DECK 04 · WEEK 4

Programs & Frontend

This is the heaviest week: state machines, cross-program invocations, and a React app that signs real transactions on devnet.

THREE EXERCISES, ONE DAPP ARC

Build logic, compose programs, then put a browser interface on top.

Exercise 5

Proposal state machine with guarded transitions: Draft → Active → Closed.

STATE MACHINE

Exercise 6

Tip jar using CPI to System Program with both `invoke` and `invoke_signed`.

CPI + PDA AUTH

Exercise 7

React frontend with wallet adapter, on-chain reads, transaction submission, and UI updates.

FULL DAPP FLOW

GUIDANCE LEVEL

AI support is minimal this week. You own the prompting, checking, and debugging loop.

STATE MACHINES

Guard transitions.

PROPOSAL/VOTING PROGRAM

State is simple. Permission checks and transition guards are the real challenge.

REQUIRED MODEL

01 `ProposalState` enum: Draft, Active, Closed.

02 Proposal PDA stores title, state, vote counts, creator pubkey.

03 `create_proposal`, `activate`, `vote`, `close` instructions.

04 Creator-only authority for activate/close.

FAILURE TESTS

01 Vote in Draft should fail.

02 Vote in Closed should fail.

03 Non-creator activate/close should fail.

04 Duplicate vote should fail (if implemented).

WHAT TO WATCH FOR

AI usually writes the happy path. You need to enforce invalid transition rejection.

Guard logic gaps

Generated code often skips checks for Draft voting or closing closed proposals.

Voter PDA collisions

Include both proposal id and voter pubkey in vote PDA seeds to avoid collisions.

Opaque errors

Use custom `#[error_code]` variants so failed transitions are explainable.

String space math

Remember string allocation includes 4-byte length prefix plus max bytes.

DEPLOY + RUN AGAINST DEVNET

```
anchor test --provider.cluster devnet
```

CPI & COMPOSABILITY

Call other programs.

INVOKE · INVOKE_SIGNED

TWO SIGNER PATTERNS

Same transfer instruction, different authority source.

`invoke` for deposit

User transfers their own SOL into vault PDA. User signs transaction.

USER SIGNER

`invoke_signed` for withdraw

Vault PDA sends SOL out. Program provides seeds + bump so PDA acts as signer.

PDA SIGNER

PDA DESIGN

Use vault seeds like `["vault", owner_wallet]` so each owner has deterministic storage and withdraw authority.

MOST CPI FAILURES ARE METADATA FAILURES

When transfers fail, inspect account metas and signer assumptions first.

SYMPTOM	LIKELY CAUSE	FIX
Missing required signature	Using `invoke` where PDA signer is needed.	Switch to `new_with_signer` + correct seeds.
Insufficient account keys	System Program or transfer accounts omitted.	Pass sender, recipient, and system program.
Invalid seeds	Derivation and signer seed bytes differ.	Unify seed order/encoding across init and withdraw.
Vault disappears	Balance dropped below rent-exempt threshold.	Check min balance or close account intentionally.

FRONTEND INTEGRATION

Connect the UI.

WALLET · STATE · TRANSACTIONS

REACT + WALLET ADAPTER + DEVNET

Your browser app needs providers for both RPC connection and wallet signing context.

REQUIRED PACKAGES

`@solana/wallet-adapter-base`, `@solana/wallet-adapter-react`, `@solana/wallet-adapter-react-ui`, `@solana/wallet-adapter-wallets`.

PROVIDERS

Wrap app with `ConnectionProvider` and `WalletProvider` so hooks and connect UI work in all child components.

CODE DESIGN RULE

Use inline chips like `anchor build` for short fragments in prose. Use full `.code-block` only for multi-line commands/snippets.

LOGO DESIGN RULE

Use the text lockup `encode` + `CLUB` with the existing `.wordmark` / `.club` styles. Do not introduce alternate logo treatments in this deck.

SHOW REAL ON-CHAIN DATA

The UI loop: derive PDA, fetch account, decode, render.

01 Derive PDA with same seeds used by your program.

02 Fetch account data from connection.

03 Decode into program types (Anchor client or manual layout parsing).

04 Render fields: state/votes or vault balance.

ANCHOR READ PATTERN

```
const proposal = await
program.account.proposal.fetch(proposalPda);
setUiState({
  title: proposal.title,
  state: proposal.state,
  yesVotes: proposal.yesVotes,
  noVotes: proposal.noVotes
});
```

USER ACTION TO CONFIRMED TX

Build instruction, request signature, send, confirm, then refresh state.

TRANSACTION FLOW

```
await program.methods
  .vote({ yes: true })
  .accounts({ proposal: proposalPda, voter: wallet.publicKey })
  .rpc();

await refetchProposal();
```

UX requirement

Show pending/confirmed status so users understand wallet popups and network delay.

Error requirement

Map program errors to readable UI messages instead of dumping raw objects.

COMMON FRONTEND INTEGRATION FAILURES

Most issues are version mismatches and stale artifacts, not React complexity.

PROBLEM	CAUSE	ACTION
Wallet adapter type errors	Package versions out of sync.	Pin all wallet-adapter packages to same version.
Instruction decode mismatch	Stale Anchor IDL in frontend.	Run `anchor build` and copy fresh IDL.
RPC 429 / flaky reads	Devnet public endpoint rate limits.	Throttle polling or use a provider endpoint.
Unexpected API syntax drift	Mixing web3.js v1 and v2 patterns.	Choose one API style and standardize codebase.

COMPLETION RUBRIC

Treat this checklist as your demo criteria for programs plus frontend.

PROGRAMS

01 Proposal transitions enforce rules with specific custom errors.

02 CPI tip jar accepts deposits and owner-only withdrawals.

03 Tests cover success and failure cases on devnet.

FRONTEND

01 Wallet connect button works on devnet.

02 On-chain state displays correctly in UI.

03 Submit action triggers wallet signature prompt.

04 UI refreshes after confirmation and shows readable errors.

YOU NOW HAVE A WORKING DAPP SHAPE

On-chain logic + program composition + wallet-connected UI is the core stack you reuse everywhere.

State machine discipline

You model allowed transitions and reject invalid state changes explicitly.

CPI reasoning

You can explain who signs and why for `invoke` vs `invoke_signed`.

Frontend transaction flow

You can connect wallet, send signed transactions, and refresh state from chain.

Debug maturity

You trace failures through seeds, accounts, IDLs, versions, and RPC behavior.